

Asynchronous Programming with asyncio

8-Step Exercise Workbook

In these exercises, you will practice writing coroutines, scheduling concurrent tasks, aggregating results with `asyncio.gather`, applying timeouts, protecting shared state with locks, throttling concurrency with semaphores, offloading blocking calls to worker threads, and building a producer-consumer pipeline with `asyncio.Queue`.

ACTIVITY 1

Exercise 1: Simple Async Greeting

Objective

Declare a basic coroutine and run it with the asyncio event loop.

What To Practice

- Define a coroutine with `async def`.
- Use `await asyncio.sleep` to pause without blocking the loop.
- Run the coroutine with `asyncio.run`.

Guide Code Snippet

```
"""
Exercise 1: Defining and Running a Coroutine
Task: Declare a simple async greeting coroutine and run it using the asyncio loop.
"""
import asyncio

async def async_greet(name):
    print(f"Preparing greeting for {name}...")
    await asyncio.sleep(0.3)
    return f"Welcome, {name} to Async Programming!"

async def main():
    result = await async_greet("Delegate")
    print(f"Result: {result}")

if __name__ == "__main__":
    asyncio.run(main())
```

Expected Shell Output

```
Preparing greeting for Delegate...
Result: Welcome, Delegate to Async Programming!
```

ACTIVITY 2

Exercise 2: Concurrent Task Scheduling

Objective

Schedule multiple API request checkers to run in parallel using `asyncio.create_task`.

What To Practice

- Create a Task for each coroutine so it starts running immediately.
- Await each Task afterwards to collect its result.
- Compare the total time against running the calls one after another.

Guide Code Snippet

```
"""
Exercise 2: Concurrent Task Scheduling
Task: Wrap multiple instances of a slow coroutine in background Tasks to execute in parallel.
"""
import asyncio
import time

async def fetch_api_endpoint(endpoint, delay):
    print(f"Polling {endpoint}...")
    await asyncio.sleep(delay)
    print(f"Polled {endpoint} successfully.")
    return f"{endpoint}_data"

async def main():
    start = time.perf_counter()

    task1 = asyncio.create_task(fetch_api_endpoint("/users", 0.5))
    task2 = asyncio.create_task(fetch_api_endpoint("/products", 0.2))
    task3 = asyncio.create_task(fetch_api_endpoint("/metrics", 0.4))

    r1, r2, r3 = await task1, await task2, await task3

    elapsed = time.perf_counter() - start
    print(f"Results: {r1}, {r2}, {r3}")
    print(f"Completed in {elapsed:.2f} seconds.")

if __name__ == "__main__":
    asyncio.run(main())
```

Expected Shell Output

```
Polling /users...
Polling /products...
Polling /metrics...
Polled /products successfully.
Polled /metrics successfully.
Polled /users successfully.
Results: /users_data, /products_data, /metrics_data
Completed in 0.50 seconds.
```

ACTIVITY 3

Exercise 3: Bulk Gathering of Awaitables

Objective

Orchestrate a list of scraper-style coroutines with `asyncio.gather`.

What To Practice

- Build a list or generator of coroutine calls.
- Pass them into `asyncio.gather` using the unpacking operator (*).
- Confirm the results come back in the same order they were submitted.

Guide Code Snippet

```
"""
Exercise 3: Bulk Gathering of Awaitables
Task: Use asyncio.gather to aggregate dynamic counts of concurrent API queries.
"""
import asyncio

async def fetch_product_inventory(product_id):
    print(f"Fetching stock levels for ID: {product_id}...")
    await asyncio.sleep(0.3)
    return {product_id: 10 + (product_id % 5)}

async def main():
    product_ids = [101, 102, 103, 104, 105]

    results = await asyncio.gather(
        *(fetch_product_inventory(pid) for pid in product_ids)
    )

    print(f"Gathered inventory counts: {results}")

if __name__ == "__main__":
    asyncio.run(main())
```

Expected Shell Output

```
Fetching stock levels for ID: 101...
Fetching stock levels for ID: 102...
Fetching stock levels for ID: 103...
Fetching stock levels for ID: 104...
Fetching stock levels for ID: 105...
Gathered inventory counts: [{101: 11}, {102: 12}, {103: 13}, {104: 14}, {105: 10}]
```

ACTIVITY 4

Exercise 4: Bounded Web Requests with Timeouts

Objective

Use `asyncio.wait_for` to prevent the event loop from hanging on a slow call.

What To Practice

- Wrap the slow coroutine call in `asyncio.wait_for` with a timeout value.
- Catch `asyncio.TimeoutError` in a try-except block.
- Log a fallback message instead of letting the program crash.

Guide Code Snippet

```
"""
Exercise 4: Bounded Web Requests with Timeouts
Task: Apply a strict timeout boundary to a lagging query, handling errors gracefully.
"""
import asyncio

async def download_heavy_archive():
    print("Beginning massive archive download stream...")
    await asyncio.sleep(2.5) # Simulate network drag
    return "ARCHIVE_BYTES"

async def main():
    try:
        data = await asyncio.wait_for(download_heavy_archive(), timeout=1.0)
        print(f"Success: {data}")
    except asyncio.TimeoutError:
        print("Network boundary exceeded! Download timed out. Falling back to local mirror.")

if __name__ == "__main__":
    asyncio.run(main())
```

Expected Shell Output

```
Beginning massive archive download stream...
Network boundary exceeded! Download timed out. Falling back to local mirror.
```

ACTIVITY 5

Exercise 5: State Synchronization with Locks

Objective

Eliminate race hazards on a shared dictionary using `asyncio.Lock`.

What To Practice

- Create a single `asyncio.Lock` shared by every worker.
- Wrap the read-modify-write section in `async` with lock.
- Run several workers concurrently and confirm the final count is correct.

Guide Code Snippet

```
"""
Exercise 5: State Synchronization with Lock
Task: Safeguard a concurrent shared metrics dict from race conditions using a Lock.
"""
import asyncio

shared_metrics = {"page_views": 0}
metrics_lock = asyncio.Lock()

async def record_page_view(user_id):
    global shared_metrics
    print(f"User {user_id} browsing page...")
    async with metrics_lock:
        current = shared_metrics["page_views"]
        await asyncio.sleep(0.05) # Intercept and force context switch
        shared_metrics["page_views"] = current + 1

async def main():
    await asyncio.gather(*(record_page_view(i) for i in range(10)))
    print(f"Expected page views: 10 | Actual recorded views: {shared_metrics['page_views']}")

if __name__ == "__main__":
    asyncio.run(main())
```

Expected Shell Output

```
User 0 browsing page...
User 1 browsing page...
... (remaining worker logs)
Expected page views: 10 | Actual recorded views: 10
```

ACTIVITY 6

Exercise 6: Throttling Concurrency with Semaphores

Objective

Restrict parallel API requests to 2 concurrent slots using `asyncio.Semaphore`.

What To Practice

- Create an `asyncio.Semaphore` with a capacity of 2.
- Acquire it with `async` with inside the throttled coroutine.
- Launch 5 concurrent requests and verify only 2 run at any one time.

Guide Code Snippet

```
"""
Exercise 6: Throttling Concurrency with Semaphores
Task: Throttle 5 concurrent operations so that at most 2 run in parallel.
"""
import asyncio
import time

api_semaphore = asyncio.Semaphore(2)

async def call_restricted_api(client_id):
    async with api_semaphore:
        print(f"Client {client_id} calling limited endpoint...")
        await asyncio.sleep(0.4)
        print(f"Client {client_id} request complete.")

async def main():
    start = time.perf_counter()
    await asyncio.gather(*(call_restricted_api(i) for i in range(5)))
    elapsed = time.perf_counter() - start
    print(f"Throttled execution complete in {elapsed:.2f} seconds.")

if __name__ == "__main__":
    asyncio.run(main())
```

Expected Shell Output

```
Client 0 calling limited endpoint...
Client 1 calling limited endpoint...
Client 0 request complete.
Client 1 request complete.
Client 2 calling limited endpoint...
... (remaining slots)
Throttled execution complete in 1.20 seconds.
```

ACTIVITY 7

Exercise 7: Offloading Blocking Synchronous Tasks

Objective

Run a heavy, CPU-bound calculation safely in a worker thread with `asyncio.to_thread`.

What To Practice

- Identify the blocking synchronous function.
- Offload it with `asyncio.to_thread` so it runs in a separate thread.
- Confirm another coroutine keeps ticking on the main thread while it runs.

Guide Code Snippet

```
"""
Exercise 7: Offloading Blocking Synchronous Tasks
Task: Execute a blocking CPU calculation safely in an executor thread without stalling loop steps.
"""
import asyncio
import time

def sync_factorial_calculation(number):
    print(f"Factorial calculation started for {number}...")
    total = 1
    for i in range(1, number + 1):
        total *= i
    time.sleep(1.0) # Blocking sleep simulating heavy processing calculations
    print(f"Factorial calculation for {number} complete.")
    return total

async def async_reporter():
    for i in range(5):
        print(f"Async reporter ticking: {i}...")
        await asyncio.sleep(0.2)

async def main():
    start = time.perf_counter()

    calc_task = asyncio.create_task(asyncio.to_thread(sync_factorial_calculation, 100))
    report_task = asyncio.create_task(async_reporter())

    await report_task
    result = await calc_task

    elapsed = time.perf_counter() - start
    print(f"Calculation Result: {result}")
    print(f"Concurrent workflow complete in {elapsed:.2f} seconds.")

if __name__ == "__main__":
    asyncio.run(main())
```

Expected Shell Output

```
Async reporter ticking: 0...
Factorial calculation started for 100...
Async reporter ticking: 1...
Async reporter ticking: 2...
Async reporter ticking: 3...
```

Async reporter ticking: 4...
Factorial calculation for 100 complete.
Concurrent workflow complete in 1.00 seconds.

ACTIVITY 8

Exercise 8: Producer-Consumer Pipeline

Objective

Build a complete async ETL-style pipeline using `asyncio.Queue` and a `Semaphore`.

What To Practice

- Declare a bounded `asyncio.Queue` to hold work items.
- Have the producer place items with `await queue.put`.
- Have a background consumer retrieve items with `await queue.get`, process them under a `Semaphore`, and call `queue.task_done`.
- `await queue.join` in main, then cancel the consumer once work has converged.

Guide Code Snippet

```
"""
Exercise 8: Capstone - Producer-Consumer Pipeline with Semaphore Rate-Limiter
Task: Build an end-to-end asynchronous pipeline where producers generate raw API payloads,
an asyncio.Queue pools them, and consumers process them under a Semaphore rate limit.
"""
import asyncio

# Rate limit: max 2 active processing tasks concurrently
process_semaphore = asyncio.Semaphore(2)

async def raw_data_producer(queue, batch_size):
    for i in range(1, batch_size + 1):
        await asyncio.sleep(0.05) # Simulate payload ingestion
        payload = f"raw_record_{i}"
        print(f"Producer pushed {payload} into queue.")
        await queue.put(payload)
    print("Producer has completed ingestion.")

async def raw_data_consumer(queue):
    while True:
        item = await queue.get()
        async with process_semaphore:
            print(f"Consumer started parsing {item}...")
            await asyncio.sleep(0.2) # Simulate work
            print(f"Consumer finished parsing {item}.")
        queue.task_done()

async def main():
    pipeline_queue = asyncio.Queue(maxsize=3)

    consumer_task = asyncio.create_task(raw_data_consumer(pipeline_queue))

    await raw_data_producer(pipeline_queue, 6)

    await pipeline_queue.join()
    print("Pipeline work has converged. Shutting down consumer background loop.")
    if consumer_task:
        consumer_task.cancel()

if __name__ == "__main__":
```

```
asyncio.run(main())
```

Expected Shell Output

```
Producer pushed raw_record_1 into queue.  
Producer pushed raw_record_2 into queue.  
Consumer started parsing raw_record_1...  
Consumer started parsing raw_record_2...  
Producer pushed raw_record_3 into queue.  
Consumer finished parsing raw_record_1.  
Consumer started parsing raw_record_3...  
... (converged logs)  
Pipeline work has converged. Shutting down consumer background loop.
```

Verification Guidelines: run each script directly, observe the printed output against the expected shell output above, then experiment by changing the delay, queue size, or concurrency limits to see how the timing and ordering change.